SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

Name of the Student	K.Neeha
Reg. No	**192425245**

COURSE NAME: Deep Learning COURSE CODE: MLA0403 FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), MiniBatch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

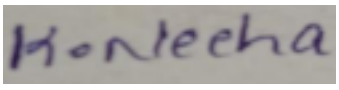
Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I K.Neeha(192425245) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: Name of the student: P. Hithaishi QUESTIONS:05 Total Marks:25

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:
Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify

with reasoning.

Answer:

Gradient Descent minimizes the loss function by updating model parameters iteratively:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

where:

- θ = model parameters
- η = learning rate
- $L(\theta)$ = loss function

Effect of High Initial Learning Rate

A high initial learning rate allows the optimizer to move rapidly toward the minimum, reducing training time during the early stages.

Advantages:

- Faster exploration of the parameter space.
- Escapes shallow local regions and flat plateaus.
- Reduces the number of iterations required initially.

Effect of Learning Rate Decay

As training progresses, gradually reducing the learning rate enables smaller and more stable updates.

Benefits:

- Prevents overshooting near the optimum.
- Improves convergence accuracy.
- Produces a smoother optimization trajectory.

When It Outperforms a Constant Learning Rate This

strategy performs better when:

- The loss function is convex.
- The initial parameters are far from the optimum.
- Large early updates speed up optimization.
- Small later updates refine the solution.

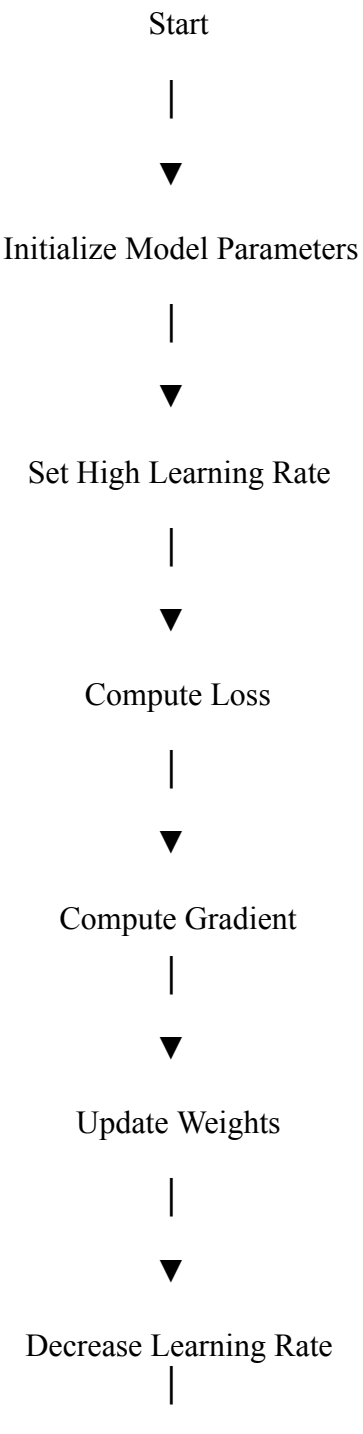
Compared to a constant learning rate:

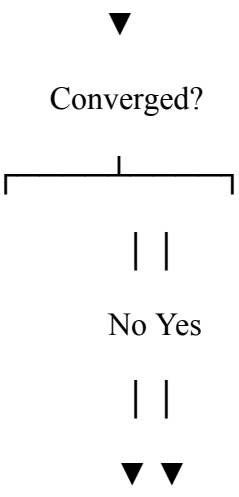
• A constant high learning rate may never converge. •
A constant low learning rate converges slowly.
• A decaying learning rate combines both speed and stability.

Risks

• Extremely high initial learning rate may overshoot the optimum. •
Training may oscillate around the minimum.
• If decay is too aggressive, learning stops prematurely. • Poor
decay schedules may result in suboptimal convergence.

Flowchart:





Conclusion

Using a high initial learning rate followed by gradual decay provides faster convergence initially and higher accuracy later. When properly tuned, it achieves better optimization than a constant learning rate by balancing exploration and precise convergence.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:
Derive the Maximum Likelihood Estimator for μ, and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases

What does this imply about the robustness of MLE?

Answer:

Assume observations

$$\text{□□}_1, \text{□□}_2, \dots, \text{□□}_{\text{□□}} \sim \text{□□}(\text{□□}, \text{□□}^2)$$

where variance □□^2 is known.

Likelihood Function

$$\text{□□}(\mu) = \prod_{\text{□□}=1}^{\text{□□}} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2} \right)$$

Taking the logarithm,

$$\log L(\mu) = -\frac{}{2}\log(2\text{□□□□□□}_2) - \frac{\text{□□}}{2\sigma^2}\sum_{\text{□□}=1}(x_i - \mu)^2 1$$

Differentiate with respect to □□:

$$\frac{\text{□□}}{\text{□□□□ □□}} \frac{\text{□□}}{1} \sum$$

Set equal to zero:

$$\sum \text{□□}_{\text{□□}} - \text{□□□□} = 0$$

Therefore,

$$\boxed{\text{□□} = \frac{1}{\text{□□}} \sum_{\text{□□}=1}^{\text{□□}} \text{□□}_{\text{□□}}}$$

Thus, the MLE of the mean is the **sample mean**.

Effect of Outliers

The sample mean is highly sensitive to extreme values.
A single large outlier shifts the average significantly, causing the estimated mean to move away from the true value.
Hence, MLE is **not robust to outliers**.

Effect of Increasing Sample Size As

the number of observations increases:

- Estimation variance decreases.
- The sample mean approaches the true population mean.
- Accuracy improves due to the Law of Large Numbers.

Thus, the estimator becomes more reliable.

Robustness of MLE Advantages:

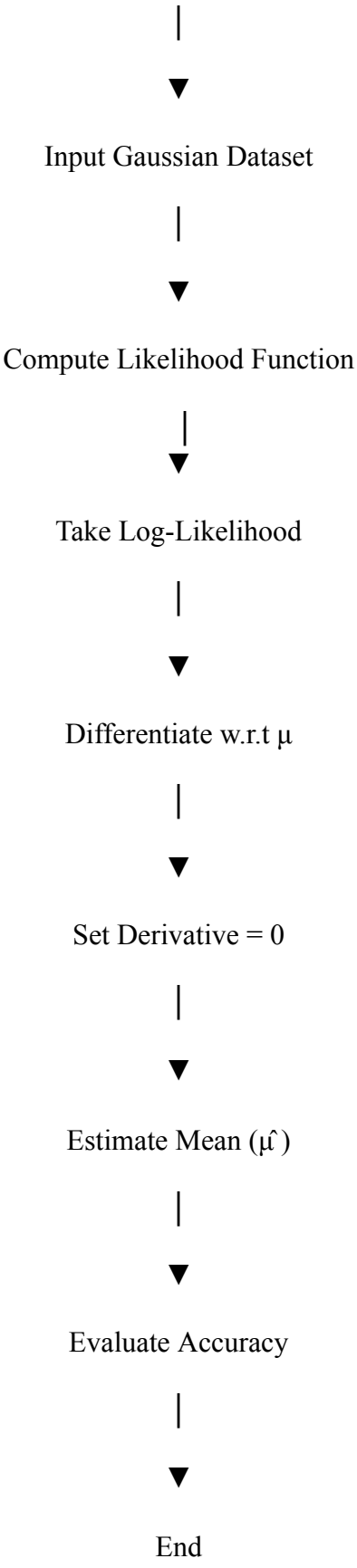
- Unbiased estimator.
- Efficient for Gaussian data.
- Consistent as sample size increases.

Limitations:

- Sensitive to outliers.
- Performance deteriorates if Gaussian assumptions are violated.

Flowchart:

Start



Conclusion

MLE provides an optimal estimate for normally distributed data with large samples but lacks robustness when outliers are present.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:
Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer:

The dataset contains:

- 10,000 features
- 500 training samples

Since the number of features greatly exceeds the number of samples, this is known as a high dimensional dataset.

Challenges

- Overfitting due to excessive model complexity.
- Curse of dimensionality.
- High computational cost.
- Increased noise and redundant features.

Proposed Machine Learning Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize features.

Reason:

Improves optimization and ensures equal feature scaling.

Step 2: Feature Selection Methods:

- Chi-Square
- Mutual Information
- LASSO

Reason:

Removes irrelevant and redundant features, reducing model complexity.

Step 3: Dimensionality Reduction Use

PCA.

Reason:
Transforms correlated variables into fewer independent principal components while preserving most information.

Step 4: Regularization Apply

L1 or L2 regularization.

Reason:
Penalizes large weights and prevents overfitting.

Step 5: Model Training Suitable

algorithms:

- Support Vector Machine (Linear SVM)
- Logistic Regression
- Random Forest (with controlled depth)

Reason:
These models perform well with high-dimensional data.

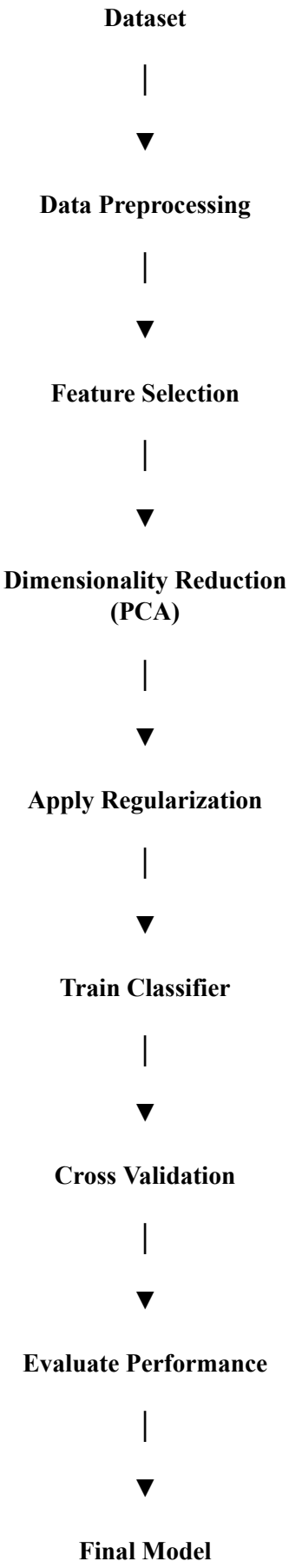
Step 6: Cross Validation Use

K-Fold Cross Validation.

Reason:
Provides reliable estimation of model performance.

How Overfitting is Reduced

- Feature selection removes unnecessary variables. •
  PCA reduces dimensionality.
- Regularization limits model complexity.
- Cross-validation ensures good generalization.


Dataset
Data Preprocessing
Feature Selection
Dimensionality Reduction
(PCA)
Apply Regularization
Train Classifier
Cross Validation
Evaluate Performance
Final Model

Conclusion

Combining feature selection, dimensionality reduction, regularization, and cross-validation produces a model that is computationally efficient and less prone to overfitting.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:
Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

A Multilayer Perceptron (MLP) consists of an input layer, hidden layer(s), and an output layer. Each

hidden neuron computes

$$\text{□□} = \text{□□}(\text{□□□□} + \text{□□})$$

where □□is the sigmoid activation function.

Why More Hidden Neurons Increase Representational Power

Each neuron learns a different nonlinear feature.

Increasing hidden neurons enables the network to:

- Learn more complex patterns.
- Approximate highly nonlinear functions.
- Represent complicated decision boundaries.

According to the Universal Approximation Theorem, a sufficiently large hidden layer can approximate any continuous function.

Why More Neurons Do Not Always Improve Performance Adding

neurons also increases:

- Number of trainable parameters.
- Computational complexity.
- Memory usage.
- Risk of memorizing training data.

Overfitting

Large networks may fit training noise instead of meaningful patterns.

Consequences:

- Very high training accuracy.
- Poor testing accuracy.
- Weak generalization.

Generalization

A well-generalized model performs well on unseen data.
Generalization improves through:

- Regularization • Dropout
- Early stopping
- Sufficient training data

Conclusion

More hidden neurons increase learning capacity but also increase overfitting risk. Optimal performance depends on balancing network complexity with generalization.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:
Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

The curse of dimensionality refers to problems that arise when the number of features becomes extremely large.

Impact on Gradient Updates During backpropagation:

- More parameters must be updated.
- Gradients become noisy.
- Some gradients may vanish or explode.
- Training becomes unstable.

Impact on SGD Convergence

SGD updates parameters using one or a few samples.

In high dimensions:

- Search space becomes extremely large.
- Gradient directions become less informative.
- More iterations are required.
- Convergence becomes slower. **Impact on Model Generalization** High-dimensional models tend to:
- Memorize training data.
- Capture noise instead of useful patterns.
- Produce high variance.
- Perform poorly on unseen data.

Techniques to Overcome These Issues

1. Dimensionality Reduction (PCA) Reason:

- Reduces feature space.
- Removes redundancy.
- Speeds up optimization.
- Improves convergence. **2. Regularization (L1/L2)** Reason:
- Penalizes large weights.
- Prevents overfitting.
- Produces smoother models.
- Improves generalization.

Additional Techniques

- Dropout to reduce neuron co-adaptation.
- Batch Normalization for stable gradients.
- Adam optimizer for adaptive learning rates.
- Feature Selection to remove irrelevant variables.
- Early Stopping to avoid over-training.

Analytical Justification

Reducing dimensions decreases parameter complexity, leading to more stable gradients and faster SGD convergence. Regularization constrains weight magnitudes, lowering variance and improving the model's ability to generalize to unseen data.

Conclusion

The curse of dimensionality negatively affects gradient updates, slows SGD convergence, and increases overfitting. Applying dimensionality reduction and regularization significantly improves training stability, convergence speed, and predictive performance.